Lab 10: Simulating a Virtual Memory and CPU Subsystem with Demand Paging and Advanced Clock Replacement

Santiago Soto (M00121124)
CS375 – Operating Systems

November 6, 2026

Abstract

This lab builds a C++ simulator of a demand-paged virtual memory system working together with a CPU scheduler. Every memory reference in a trace goes through a configurable LRU TLB, a two-level page table and an enhanced Clock replacement policy that uses reference and modified bits. A thread-level scheduler runs the trace under FCFS, Round Robin, Priority and a three-queue MLFQ with aging. The simulator logs every access, generates text and `matplotlib` timeline graphs, and reports the TLB hit rate, page-fault rate, context and thread switches, and a fairness index. The experiments show that the scheduling policy changes memory performance as much as the hardware parameters do. Policies that interleave many threads destroy TLB and frame locality, and a TLB larger than the set of resident frames brings no further benefit.

Contents

1 Building and Running **3**

2 Design and Implementation **4**
2.1 Two-level page table and page-fault handling 4
2.2 Configurable TLB with LRU 5
2.3 Advanced Clock with modified bits 6
2.4 Thread simulation and scheduling (FCFS, RR, Priority, MLFQ) 7

3 Modified Trace and Access Patterns **9**

4 Results **10**
4.1 Summary table of performance metrics 10
4.2 Timeline graphs 11
4.3 Access patterns, TLB size and frame count 14

5 Analysis **16**
5.1 Scheduling policy vs. memory performance 16
5.2 Impact of memory access patterns 17
5.3 Impact of TLB size 18
5.4 Conclusion 18

A Generated Files **19**

1 Building and Running

The project builds with a single `make`. The simulator binary is `bin/vmsim`. With no arguments it runs all four scheduling policies on `input/trace.txt`.

```
make # build bin/vmsim from src/
./bin/vmsim # all four policies, TLB = 8, 16 frames
./bin/vmsim --policy mlfq --tlb 4 --verbose # one policy, full log on screen
./bin/vmsim --config config/sim.conf # settings from a config file
make experiments # every pattern / policy / TLB size / frame count
make plots # PNG timelines + comparison graphs
```

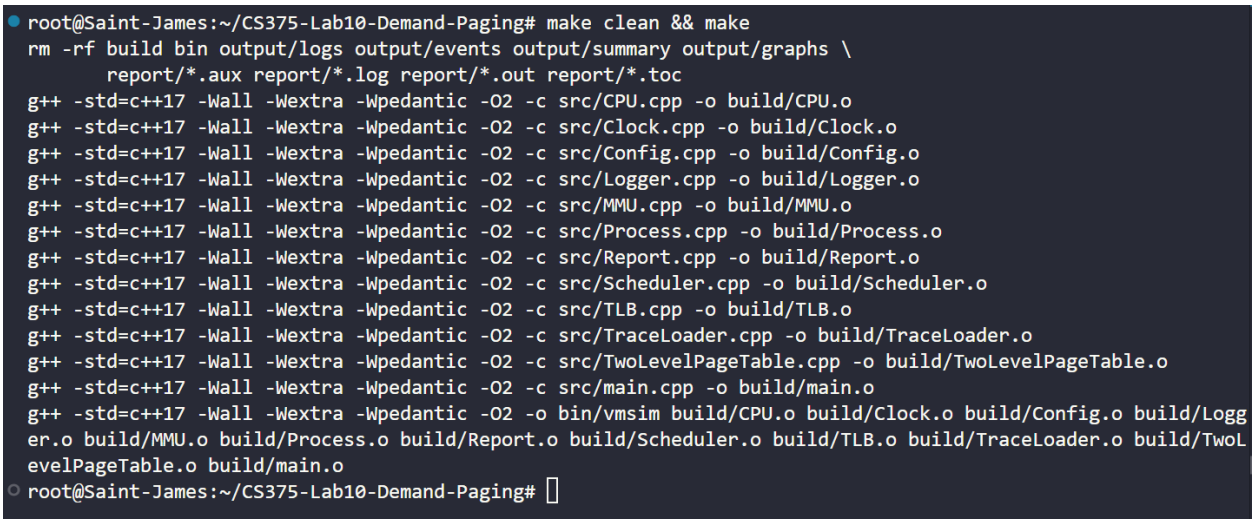


Figure 1: Building the simulator with `make`.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# ./bin/vmsim --help
Usage: ./bin/vmsim [options]

Virtual memory + CPU scheduling simulator (CS375 Lab 10)

Input / output:
  --trace FILE        trace file to simulate        (default input/trace.txt)
  --config FILE       load key=value settings from a file (e.g. config/sim.conf)
  --out DIR           output directory              (default output)
  --tag NAME          label for output files        (default: trace file name)
  --summary FILE      append one CSV row per policy to FILE

Memory system:
  --tlb N             TLB entries, LRU eviction     (default 8, 0 = no TLB)
  --frames N          physical frames               (default 16)
  --page-size N       bytes per page, power of two  (default 256)
  --address-bits N    virtual address width         (default 16)
  --tlb-flush         flush the TLB on every context switch (untagged TLB)

Scheduling:
  --policy LIST       fcfs, rr, priority, mlfq or all; comma separated (default all)
  --quantum N         RR/Priority quantum and MLFQ Q0 quantum (default 4)
  --aging N           MLFQ aging threshold in ticks (default 30)

Console:
  --verbose           print every log line to the terminal as well
  --quiet             print only the final comparison table
  --no-timeline       do not print the ASCII timeline
  --timeline-ticks N  ticks of timeline to print on screen (default 200, 0 = all)
  --timeline-width N  ticks per timeline row (default 100)
  -h, --help          show this help

Examples:
  ./bin/vmsim                                 # all four policies on input/trace.txt
  ./bin/vmsim --policy mlfq --tlb 4 --verbose   # one policy, small TLB, full log
  ./bin/vmsim --config config/sim.conf          # settings from a file
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 2: Command-line options. The TLB size, frame count, page size and policy are all configurable.

2 Design and Implementation

File	Responsibility
`main.cpp`	driver: parses options, runs each policy, writes logs, CSVs and the summary
`Process.h/cpp`	process with threads; per-thread access streams, bursts and priorities
`CPU.h/cpp`	executes one cycle; counts context vs. thread switches
`Scheduler.h/cpp`	FCFS, RR (q=4), Priority, MLFQ (3 queues, aging)
`TLB.h/cpp`	configurable-size TLB with LRU eviction and invalidation
`TwoLevelPageTable.h/cpp`	outer/inner page tables, on-demand second-level tables
`Clock.h/cpp`	enhanced second-chance Clock with R and M bits
`MMU.h/cpp`	translation path TLB → page walk → page fault
`TraceLoader`, `Config`, `Logger`, `Report`	trace parsing, options, log file, tables and timelines

Table 1: Source files (all in `src/`).

2.1 Two-level page table and page-fault handling

Virtual addresses are 16 bits with 256-byte pages, so every process has 256 virtual pages. The virtual page number is split evenly into an outer and an inner index:

bits 15–12	bits 11–8	bits 7–0	example `0x1A43`
outer index (4)	inner index (4)	offset (8)	page 26 = outer 1, inner 10, offset `0x43`

`firstLevel` maps `(pid, outer)` to the id of a second-level table, and `secondLevel` maps `(table id, inner)` to a frame number, where −1 means not present. Second-level tables are allocated only when the first page in their range faults in. In the FCFS run of `trace.txt`, 24 second-level tables (384 entries) were allocated, compared with 1024 entries for four flat page tables. That is the space saving that hierarchical paging is designed for.

Every access follows the same path in `MMU::access()`: (1) TLB lookup; (2) on a miss, walk both levels; (3) if the page is not present, a page fault occurs. Clock selects a frame. If the victim is dirty it is written back, and the victim's page-table entry and TLB entry are both invalidated. Then both levels are updated for the new page and the TLB is refilled. Invalidating the TLB entry on eviction matters: without it, a TLB hit could return a frame that now belongs to another page.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep -m 15 -E "PAGE FAULT|page table hit" output/logs/trace_fcfs.log
  [t=    0] P1.T1  R 0x0043  page 0 (outer 0, inner 0) off 0x43 | TLB MISS | PAGE FAULT -> frame 0 (free frame) | PA 0x0043
  [t=    2] P1.T1  W 0x1000  page 16 (outer 1, inner 0) off 0x00 | TLB MISS | PAGE FAULT -> frame 1 (free frame) | PA 0x0100
  [t=   10] P1.T1  R 0x1100  page 17 (outer 1, inner 1) off 0x00 | TLB MISS | PAGE FAULT -> frame 2 (free frame) | PA 0x0200
  [t=   18] P1.T1  W 0x1200  page 18 (outer 1, inner 2) off 0x00 | TLB MISS | PAGE FAULT -> frame 3 (free frame) | PA 0x0300
  [t=   26] P1.T1  R 0x1300  page 19 (outer 1, inner 3) off 0x00 | TLB MISS | PAGE FAULT -> frame 4 (free frame) | PA 0x0400
  [t=   34] P1.T1  R 0x1400  page 20 (outer 1, inner 4) off 0x00 | TLB MISS | PAGE FAULT -> frame 5 (free frame) | PA 0x0500
  [t=   43] P1.T2  W 0x3000  page 48 (outer 3, inner 0) off 0x00 | TLB MISS | PAGE FAULT -> frame 6 (free frame) | PA 0x0600
  [t=   51] P1.T2  R 0x3100  page 49 (outer 3, inner 1) off 0x00 | TLB MISS | PAGE FAULT -> frame 7 (free frame) | PA 0x0700
  [t=   59] P1.T2  W 0x3200  page 50 (outer 3, inner 2) off 0x00 | TLB MISS | PAGE FAULT -> frame 8 (free frame) | PA 0x0800
  [t=   67] P2.T1  R 0x0210  page 2 (outer 0, inner 2) off 0x10 | TLB MISS | PAGE FAULT -> frame 9 (free frame) | PA 0x0910
  [t=   68] P2.T1  R 0xB782  page 183 (outer 11, inner 7) off 0x82 | TLB MISS | PAGE FAULT -> frame 10 (free frame) | PA 0x0A82
  [t=   69] P2.T1  R 0x96CD  page 150 (outer 9, inner 6) off 0xCD | TLB MISS | PAGE FAULT -> frame 11 (free frame) | PA 0x0BCD
  [t=   70] P2.T1  W 0x63CC  page 99 (outer 6, inner 3) off 0xCC | TLB MISS | PAGE FAULT -> frame 12 (free frame) | PA 0x0CCC
  [t=   71] P2.T1  R 0xA9D3  page 169 (outer 10, inner 9) off 0xD3 | TLB MISS | PAGE FAULT -> frame 13 (free frame) | PA 0x0DD3
  [t=   72] P2.T1  W 0x79DD  page 121 (outer 7, inner 9) off 0xDD | TLB MISS | PAGE FAULT -> frame 14 (free frame) | PA 0x0EDD
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep "page table hit, frame" output/logs/trace_fcfs.log
  [t=  101] P2.T1  R 0xDBE2  page 219 (outer 13, inner 11) off 0xE2 | TLB MISS | page table hit, frame 12 | PA 0x0CE2
  [t=  114] P2.T1  R 0x7419  page 116 (outer 7, inner 4) off 0x19 | TLB MISS | page table hit, frame 0 | PA 0x0019
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep -A5 "Two-level page table" output/logs/trace_fcfs.log
    Two-level page table (4 outer bits + 4 inner bits, 256 pages per process)
      page-table walks       : 117
      page faults            : 115
      pages loaded / evicted : 115 / 99
      resident pages now     : 16
      second-level tables    : 24  (384 inner entries allocated vs 1024 for flat tables)
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 3: Per-access log: each address is split into outer/inner/offset, and the log shows TLB misses, page-table hits and page faults. Below it are the page-table statistics.

2.2 Configurable TLB with LRU

The TLB size is set with `--tlb N` or with `tlb_size` in `config/sim.conf`. Setting it to 0 disables the TLB. Each entry is tagged with the PID and stores a `lastUsed` timestamp. A hit refreshes the timestamp, and an insert into a full TLB replaces the entry with the smallest timestamp, which is true LRU. With `--tlb-flush` the TLB is flushed on every context switch, which models an untagged TLB.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# ./bin/vmsim --tlb 2 --quiet --tag tlb2 | head -11

================================================================================================
 PERFORMANCE SUMMARY  trace=tlb2  TLB=2 entries  frames=16  page=256 B  accesses=323
================================================================================================
 Policy    TLB hit%  Fault%  Faults  WriteBk  CtxSw  ThrSw  AvgWait  AvgTurn  AvgResp  MaxWait  Fairness
 --------  --------  ------  ------  -------  -----  -----  -------  -------  -------  -------  --------
 FCFS         47.06   35.60     115       52      5      2    141.2    181.6    141.2      311     0.576
 RR           34.06   48.30     156       60     67     14    184.0    224.4     14.0      243     0.943
 Priority     36.84   34.98     113       51     42     18    145.8    186.1    108.0      274     0.715
 MLFQ         39.32   48.61     157       61     41      7    175.0    215.4     14.0      243     0.947
------------------------------------------------------------------------------------------------
root@Saint-James:~/CS375-Lab10-Demand-Paging# ./bin/vmsim --tlb 16 --quiet --tag tlb16 | head -11

================================================================================================
 PERFORMANCE SUMMARY  trace=tlb16  TLB=16 entries  frames=16  page=256 B  accesses=323
================================================================================================
 Policy    TLB hit%  Fault%  Faults  WriteBk  CtxSw  ThrSw  AvgWait  AvgTurn  AvgResp  MaxWait  Fairness
 --------  --------  ------  ------  -------  -----  -----  -------  -------  -------  -------  --------
 FCFS         64.40   35.60     115       52      5      2    141.2    181.6    141.2      311     0.576
 RR           51.70   48.30     156       60     67     14    184.0    224.4     14.0      243     0.943
 Priority     65.02   34.98     113       51     42     18    145.8    186.1    108.0      274     0.715
 MLFQ         51.39   48.61     157       61     41      7    175.0    215.4     14.0      243     0.947
------------------------------------------------------------------------------------------------
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 4: The same trace with a 2-entry TLB and with a 16-entry TLB. Only the TLB hit rate changes; the page-fault rate stays the same.

2.3 Advanced Clock with modified bits

Each frame keeps a reference bit R and a modified bit M. Loading a page sets R=1, and M=1 if the first access is a write. Every hit sets R, and a write also sets M. When memory is full, the hand searches as follows:

1. **Pass A** – one full sweep for a frame with (R=0, M=0) without changing any bits.
2. **Pass B** – one full sweep for (R=0, M=1), clearing R on every frame it passes.
3. Repeat. After pass B every R bit is 0, so the next round always finds a victim.

Clean, unreferenced pages are therefore evicted first, and a dirty page is written back only when no clean candidate exists. The trace `input/clock_demo.txt` demonstrates this with 3 frames. When page 3 arrives, the hand reaches dirty page 0 first, but clean page 1 is evicted instead. In the full `trace.txt` FCFS run, 47 of the 99 evictions were clean and needed no disk write.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# ./bin/vmsim --trace input/clock_demo.txt --frames 3 --policy fcfs --verbose --no-timeline | grep -E "^\[t|
Advanced|clean|dirty|frame "
[t=    0] ARRIVE   P1.T1  burst=7 priority=5
[t=    0] DISPATCH P1.T1  (runs to completion)
[t=    0] P1.T1  W 0x0000  page 0 (outer 0, inner 0) off 0x00 | TLB MISS | PAGE FAULT -> frame 0 (free frame) | PA 0x0000
[t=    1] P1.T1  R 0x0100  page 1 (outer 0, inner 1) off 0x00 | TLB MISS | PAGE FAULT -> frame 1 (free frame) | PA 0x0100
[t=    2] P1.T1  W 0x0200  page 2 (outer 0, inner 2) off 0x00 | TLB MISS | PAGE FAULT -> frame 2 (free frame) | PA 0x0200
[t=    3] P1.T1  R 0x0300  page 3 (outer 0, inner 3) off 0x00 | TLB MISS | PAGE FAULT -> frame 1 (evicted P1 page 1, clean) | PA 0x0100
[t=    4] P1.T1  R 0x0000  page 0 (outer 0, inner 0) off 0x00 | TLB HIT  | PA 0x0000
[t=    5] P1.T1  R 0x0400  page 4 (outer 0, inner 4) off 0x00 | TLB MISS | PAGE FAULT -> frame 2 (evicted P1 page 2, dirty -> write-back) | PA 0x0200
[t=    6] P1.T1  R 0x0500  page 5 (outer 0, inner 5) off 0x00 | TLB MISS | PAGE FAULT -> frame 1 (evicted P1 page 3, clean) | PA 0x0100
[t=    7] FINISH   P1.T1  turnaround=7 wait=0
[t=    7] IDLE     all threads finished
  Advanced Clock (3 frames)
      clean (R=0,M=0)      : 2
      dirty (write-back)   : 1
    frame  0 : P1 page   0  R=0 M=1
    frame  1 : P1 page   5  R=1 M=0
    frame  2 : P1 page   4  R=0 M=0   <- hand
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 5: Clock demo: clean pages are evicted before dirty ones. The final frame table shows the R and M bits.

2.4 Thread simulation and scheduling (FCFS, RR, Priority, MLFQ)

Each trace line carries a PID and a thread ID. A `Process` keeps a separate access stream, CPU burst (number of accesses) and priority for each thread, so the schedulable unit is a thread `(pid, tid)`. One tick is one CPU cycle, and each cycle performs one memory access. The CPU distinguishes two kinds of switch:

- **context switch** – the next thread belongs to a *different process*, so the address space changes;
- **thread switch** – the next thread belongs to the *same process*, so the address space and the TLB contents stay valid.

FCFS runs threads to completion in arrival order. **RR** uses a quantum of 4. **Priority** always runs the ready thread with the lowest priority number and shares the CPU round robin among equal priorities. Process and thread priorities come from `PRIORITY` and `THREAD` lines in the trace. **MLFQ** has three queues with quanta 4, 8 and 16. Every thread starts in Q0 and is demoted one level each time it uses its whole quantum. A thread that has waited 30 ticks in Q1 or Q2 is promoted one level (aging), which prevents starvation.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep -m 15 -E "DEMOTE|PROMOTE" output/logs/trace_mlfq.log
  [t=    4] DEMOTE   P1.T1  Q0 -> Q1 (used full quantum)
  [t=    8] DEMOTE   P1.T2  Q0 -> Q1 (used full quantum)
  [t=   12] DEMOTE   P2.T1  Q0 -> Q1 (used full quantum)
  [t=   16] DEMOTE   P3.T1  Q0 -> Q1 (used full quantum)
  [t=   20] DEMOTE   P2.T2  Q0 -> Q1 (used full quantum)
  [t=   24] DEMOTE   P3.T2  Q0 -> Q1 (used full quantum)
  [t=   28] DEMOTE   P4.T1  Q0 -> Q1 (used full quantum)
  [t=   32] DEMOTE   P4.T2  Q0 -> Q1 (used full quantum)
  [t=   38] PROMOTE  P1.T2  Q1 -> Q0 (aging: waited 30 ticks)
  [t=   40] DEMOTE   P1.T1  Q1 -> Q2 (used full quantum)
  [t=   42] PROMOTE  P2.T1  Q1 -> Q0 (aging: waited 30 ticks)
  [t=   44] DEMOTE   P1.T2  Q0 -> Q1 (used full quantum)
  [t=   46] PROMOTE  P3.T1  Q1 -> Q0 (aging: waited 30 ticks)
  [t=   48] DEMOTE   P2.T1  Q0 -> Q1 (used full quantum)
  [t=   50] PROMOTE  P2.T2  Q1 -> Q0 (aging: waited 30 ticks)
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep -A11 "Per-thread" output/logs/trace_mlfq.log
    Per-thread results (MLFQ)
      Thread    Prio  Burst  Response  Wait  Turnaround  Dispatches  TLB hit%  Faults
      --------  ----  -----  --------  ----  ----------  ----------  --------  ------
      P1.T1        2     42         0   203         245           7      73.8      10   (MLFQ: 6 demotions, 5 promotions)
      P1.T2        3     25         4   129         154           5      76.0       4   (MLFQ: 4 demotions, 3 promotions)
      P2.T1        1     65         8   217         282           8       1.5      61   (MLFQ: 5 demotions, 3 promotions)
      P3.T1        3     33        12   174         207           6      42.4      16   (MLFQ: 5 demotions, 4 promotions)
      P2.T2        1     17        16   104         121           4       5.9      16   (MLFQ: 3 demotions, 2 promotions)
      P3.T2        3     49        20   238         287           8      49.0      23   (MLFQ: 7 demotions, 5 promotions)
      P4.T1        2     80        24   243         323          10      63.8      24   (MLFQ: 7 demotions, 5 promotions)
      P4.T2        1     12        28    92         104           3      75.0       3   (MLFQ: 2 demotions, 2 promotions)

root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 6: MLFQ log: demotions after a full quantum, promotions by aging, and the per-thread table with demotion and promotion counts.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep -m 16 -E "DISPATCH|SWITCH" output/logs/trace_rr.log
[t=    0] DISPATCH P1.T1  (quantum 4)
[t=    4] DISPATCH P1.T2  (quantum 4)
[t=    4] THREAD SWITCH  -> P1.T2 (same address space)
[t=    8] DISPATCH P2.T1  (quantum 4)
[t=    8] CONTEXT SWITCH -> P2
[t=   12] DISPATCH P3.T1  (quantum 4)
[t=   12] CONTEXT SWITCH -> P3
[t=   16] DISPATCH P2.T2  (quantum 4)
[t=   16] CONTEXT SWITCH -> P2
[t=   20] DISPATCH P3.T2  (quantum 4)
[t=   20] CONTEXT SWITCH -> P3
[t=   24] DISPATCH P4.T1  (quantum 4)
[t=   24] CONTEXT SWITCH -> P4
[t=   28] DISPATCH P4.T2  (quantum 4)
[t=   28] THREAD SWITCH  -> P4.T2 (same address space)
[t=   32] DISPATCH P1.T1  (quantum 4)
root@Saint-James:~/CS375-Lab10-Demand-Paging# grep -A11 "Per-thread" output/logs/trace_rr.log
  Per-thread results (RR)
    Thread    Prio  Burst  Response  Wait  Turnaround  Dispatches  TLB hit%  Faults
    --------  ----  -----  --------  ----  ----------  ----------  --------  ------
    P1.T1        2     42         0   207         249          11      61.9      10
    P1.T2        3     25         4   153         178           7      64.0       6
    P2.T1        1     65         8   242         307          17       1.5      64
    P3.T1        3     33        12   190         223           9      36.4      17
    P2.T2        1     17        16   124         141           5       5.9      16
    P3.T2        3     49        20   229         278          13      36.7      20
    P4.T1        2     80        24   243         323          20      53.8      20
    P4.T2        1     12        28    84          96           3      75.0       3

root@Saint-James:~/CS375-Lab10-Demand-Paging# \
```

Figure 7: Thread-level scheduling under RR: thread switches (same address space) and context switches (new process), plus per-thread burst, wait and turnaround times.

3 Modified Trace and Access Patterns

`scripts/gen_traces.py` uses a fixed seed, so every result in this report can be reproduced exactly. The modified `input/trace.txt` keeps the 7 original sample lines and adds four sections: P1 *sequential* (a strided array walk, 8 accesses per page), P2 *random* (uniform over 64 KiB), P3 *localized* (90% of accesses in a 3-page hot set that moves halfway through), and P4 *mixed* (one localized thread and one sequential thread). The three pattern traces `trace_sequential/random/localized.txt` use the *same* 8 threads, burst lengths and priorities, so the access pattern is the only variable. About 30% of all accesses are writes.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# head -n 45 input/trace.txt
# ======================================================================
# CS375 Lab 10 - modified trace with varied access patterns
# Section 0: the original 7-line sample from the handout
# Section 1: P1 (both threads) SEQUENTIAL  - strided array walk
# Section 2: P2 (both threads) RANDOM      - uniform over 64 KiB
# Section 3: P3 (both threads) LOCALIZED   - 90% in a 3-page hot set
# Section 4: P4 T1 LOCALIZED, P4 T2 SEQUENTIAL (mixed process)
# Format: <pid> <thread_id> <address> <access_type> (R=read, W=write)
#         PRIORITY <pid> <prio>  /  THREAD <pid> <tid> <prio>  (lower = higher)
# ======================================================================

# ---- priorities ----
PRIORITY 1 2
PRIORITY 2 1
PRIORITY 3 3
PRIORITY 4 2
THREAD 1 2 3
THREAD 4 2 1

# ---- Section 0: original sample ----
1 1 0x0043 R
1 2 0x00A1 W
2 1 0x0210 R
3 1 0x0F0F R
1 1 0x00B0 W
2 2 0x0213 R
3 2 0x0100 W

# ---- Section 1: process 1 ----
# P1.T1: 40 sequential accesses
1 1 0x1000 W
1 1 0x1020 W
1 1 0x1040 R
1 1 0x1060 R
1 1 0x1080 W
1 1 0x10A0 R
1 1 0x10C0 R
1 1 0x10E0 R
1 1 0x1100 R
1 1 0x1120 R
1 1 0x1140 R
1 1 0x1160 R
1 1 0x1180 R
1 1 0x11A0 R
1 1 0x11C0 W
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 8: The modified `input/trace.txt`: header, priorities, the original sample and the start of the sequential section.

4 Results

Default configuration: TLB = 8 entries (LRU, tagged), 16 frames, 256-byte pages, quantum 4, aging 30. All 323 accesses of `trace.txt` take 323 ticks under every policy because there is no idle time, so the policies differ only in *order*. Even so, the order changes memory performance considerably.

4.1 Summary table of performance metrics

Policy	TLB hit %	Fault %	Faults	Write-backs	Ctx sw	Thr sw	Avg wait	Fairness
FCFS	63.78	35.60	115	52	5	2	141.2	0.576
RR	39.01	48.30	156	60	67	14	184.0	0.943
Priority	62.23	34.98	113	51	42	18	145.8	0.715
MLFQ	46.44	48.61	157	61	41	7	175.0	0.947

Table 2: `input/trace.txt` (mixed patterns), generated automatically by the simulator. Fairness is Jain's index of *burst/turnaround* over all threads (1 = perfectly fair).

Sequential

Policy	TLB hit %	Fault %	Faults	Write-backs	Ctx sw	Thr sw	Avg wait	Fairness
FCFS	87.34	12.66	40	23	3	4	135.0	0.580
RR	87.34	12.66	40	23	53	21	170.0	0.953
Priority	87.34	12.66	40	23	38	15	140.5	0.716
MLFQ	81.96	12.97	41	23	33	16	172.0	0.948

Random

Policy	TLB hit %	Fault %	Faults	Write-backs	Ctx sw	Thr sw	Avg wait	Fairness
FCFS	2.85	94.94	300	80	3	4	135.0	0.580
RR	1.90	97.78	309	81	53	21	170.0	0.953
Priority	2.85	95.57	302	84	38	15	140.5	0.716
MLFQ	1.90	97.78	309	82	33	16	172.0	0.948

Localized

Policy	TLB hit %	Fault %	Faults	Write-backs	Ctx sw	Thr sw	Avg wait	Fairness
FCFS	81.01	18.99	60	33	3	4	135.0	0.580
RR	45.89	37.66	119	42	53	21	170.0	0.953
Priority	75.95	19.62	62	31	38	15	140.5	0.716
MLFQ	54.75	34.49	109	41	33	16	172.0	0.948

Table 3: The same 8 threads with a single access pattern each.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# ./bin/vmsim --quiet

================================================================================================================
 PERFORMANCE SUMMARY  trace=trace  TLB=8 entries  frames=16  page=256 B  accesses=323
================================================================================================================
 Policy    TLB hit%  Fault%  Faults  WriteBk  CtxSw  ThrSw  AvgWait  AvgTurn  AvgResp  MaxWait  Fairness
 --------  --------  ------  ------  -------  -----  -----  -------  -------  -------  -------  --------
 FCFS         63.78   35.60     115       52      5      2    141.2    181.6    141.2      311     0.576
 RR           39.01   48.30     156       60     67     14    184.0    224.4     14.0      243     0.943
 Priority     62.23   34.98     113       51     42     18    145.8    186.1    108.0      274     0.715
 MLFQ         46.44   48.61     157       61     41      7    175.0    215.4     14.0      243     0.947
----------------------------------------------------------------------------------------------------------------
 Fault% = page faults / accesses.  Fairness = Jain's index of (burst / turnaround), 1.000 = fair.
 CtxSw = switch to another process, ThrSw = switch between threads of the same process.

 Output files:
   log     : output/trace_output.log  (all policies + summary)
   logs    : output/logs/trace_<policy>.log
   events  : output/events/trace_<policy>.csv
   summary : output/summary/trace.csv  and  output/summary/trace_table.tex
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 9: Terminal output of the performance summary for `trace.txt`.

4.2 Timeline graphs

Each tick is coloured by the outcome of that access. Grey bars show the time a thread spends ready but waiting, solid vertical lines mark context switches and dotted lines mark thread switches.

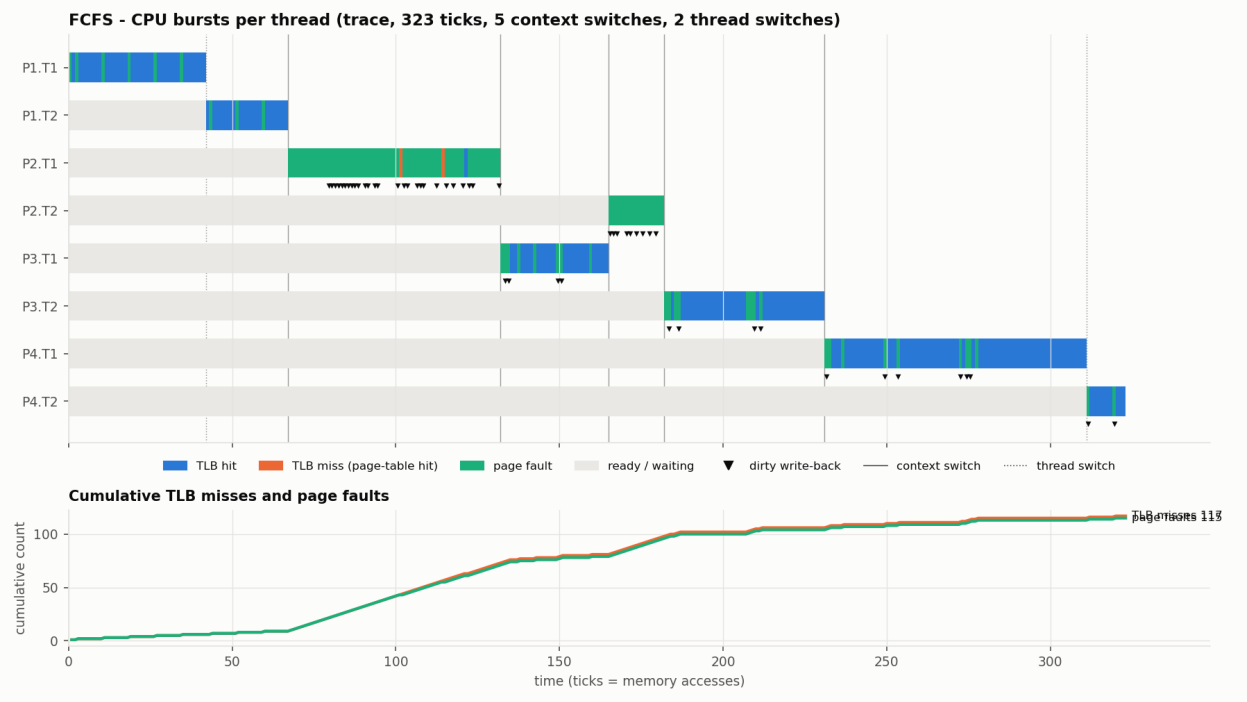


Figure 10: FCFS timeline: long uninterrupted bursts, so pages are faulted in once and then hit in the TLB.

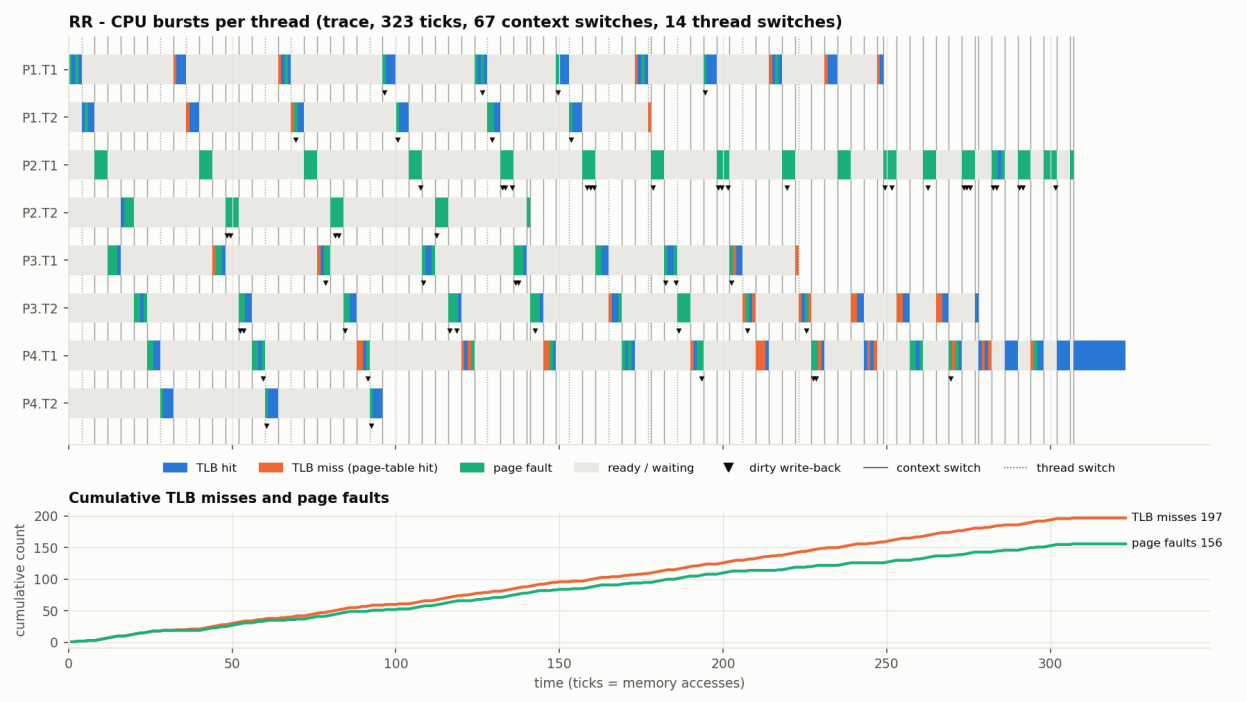


Figure 11: Round Robin timeline: a context switch every 4 ticks, with TLB misses and faults at the start of almost every quantum.



Figure 12: Priority timeline: the priority-1 threads finish first and the priority-3 threads (P3, P1.T2) wait until the end.

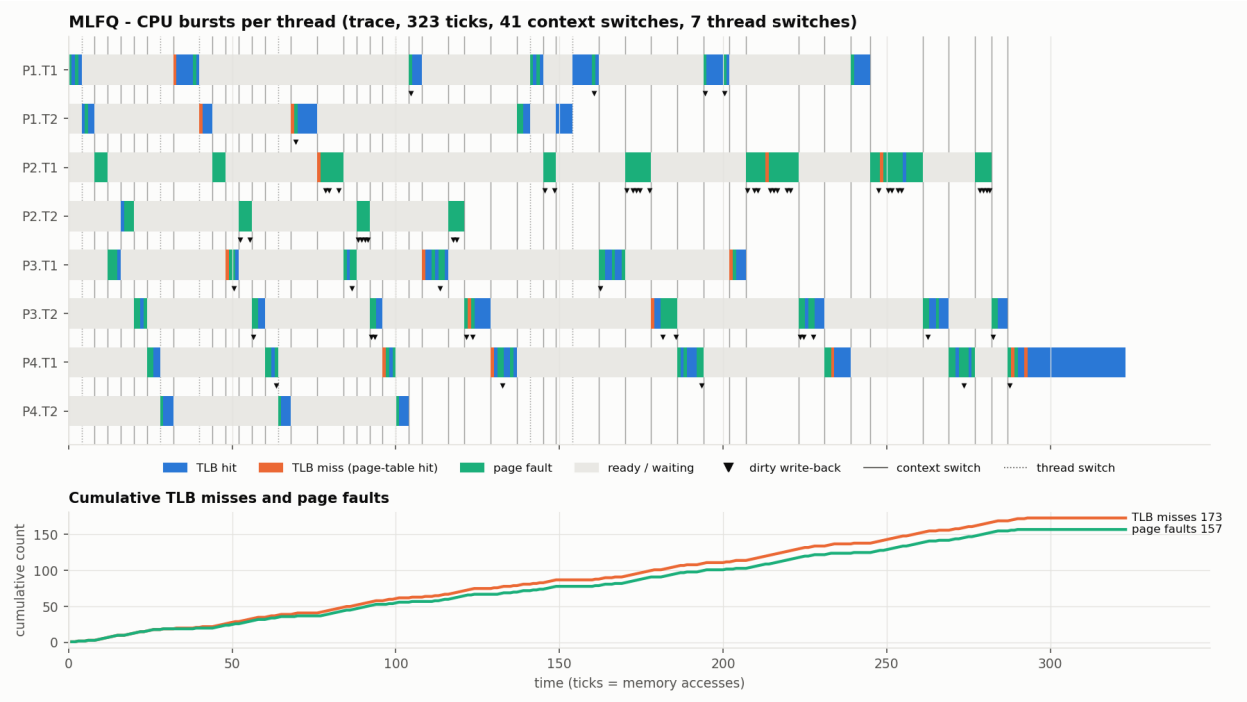


Figure 13: MLFQ timeline: quanta grow from 4 to 16 as threads are demoted, and aging promotions keep the waiting threads moving.

```
root@Saint-James:~/CS375-Lab10-Demand-Paging# sed -n '/ASCII timeline/,$p' output/logs/trace_rr.log | head -27
  ASCII timeline - RR  (ticks 0-322 of 323)
  Legend: '=' TLB hit   'm' TLB miss (page table hit)   'F' page fault   'W' page fault + dirty write-back
          '.' ready/waiting   'C' context switch   't' thread switch

  tick     0        10       20       30       40       50       60       70       80       90
           |--------|--------|--------|--------|--------|--------|--------|--------|--------|--------
  P1.T1    F=F=.........................m===.........................m=F=.........................W===
  P1.T2    ....=F==.........................m===.........................mW==.........................
  P2.T1    ........FFFF.........................FFFF.........................FFFF.....................
  P3.T1    ............FFF=.........................mFF=.........................m=WF...................
  P2.T2    ................=FFF.........................WWFF.........................FWWF................
  P3.T2    ....................FF=F.........................WW==.........................WF==............
  P4.T1    ........................FF==.........................FF=W.........................mm=W........
  P4.T2    ............................F===.........................W===.........................W===
  Switch       t   C   C   C   C   C   t   C   t   C   C   C   C   C   t   C   t   C   C   C   C   C   t   C

  tick     100      110      120      130      140      150      160      170      180      190
           |--------|--------|--------|--------|--------|--------|--------|--------|--------|--------
  P1.T1    ........................F=W=.....................W===.....................m=F=.................W===..
  P1.T2    W===........................FW==.....................W===.....................m
  P2.T1    ....FFFW........................WWFW.....................FWWW.................WFFF.................WW
  P3.T1    ........W==F.........................WWF=.....................FF==.................W==W..............
  P2.T2    ............WFFF.........................F
  P3.T2    ................WFW=.....................FWF=.....................m==F.................WFFF..........
  P4.T1    ....................m=mF.....................mmF=.....................F=F=.................m=FW......
  P4.T2
  Switch   t   C   C   C   C   C   C   t   C   C   CC   C   C   t   C   C   t   C   C   tC   C   t   C   C   C
root@Saint-James:~/CS375-Lab10-Demand-Paging#
```

Figure 14: The text-based (ASCII) timeline printed by the simulator: one row per thread, plus a row for context and thread switches.

4.3 Access patterns, TLB size and frame count

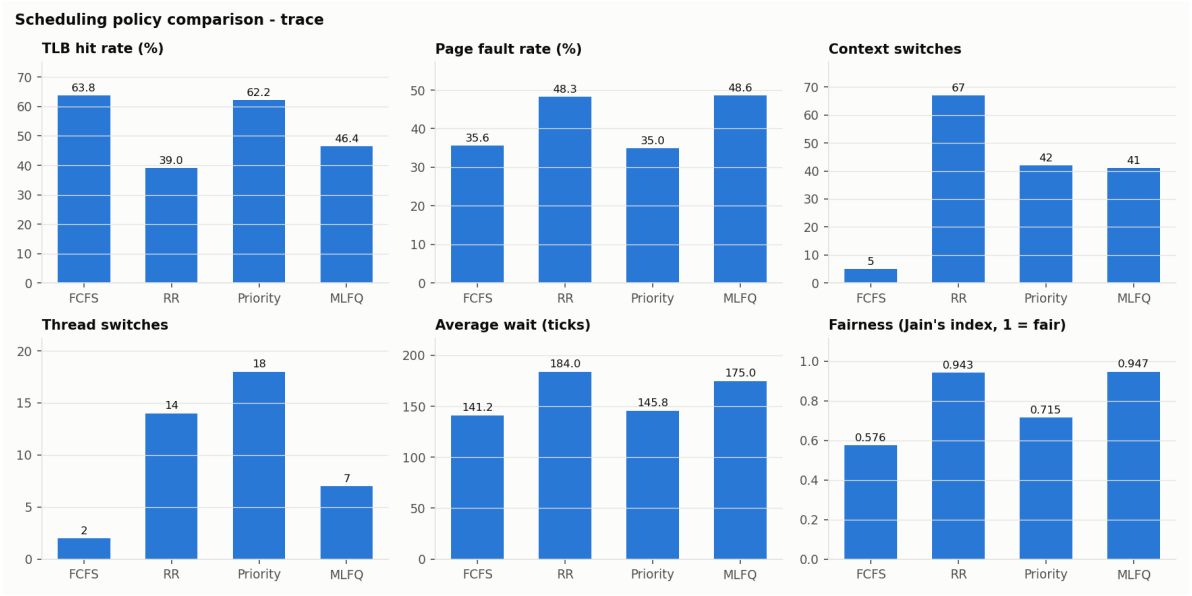


Figure 15: Policy comparison on `trace.txt`.

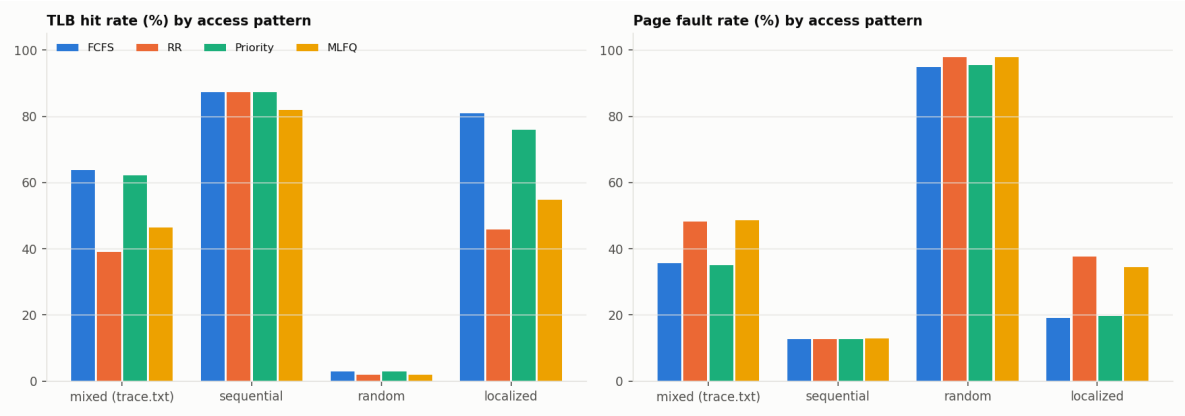


Figure 16: TLB hit rate and page-fault rate for each access pattern and policy.

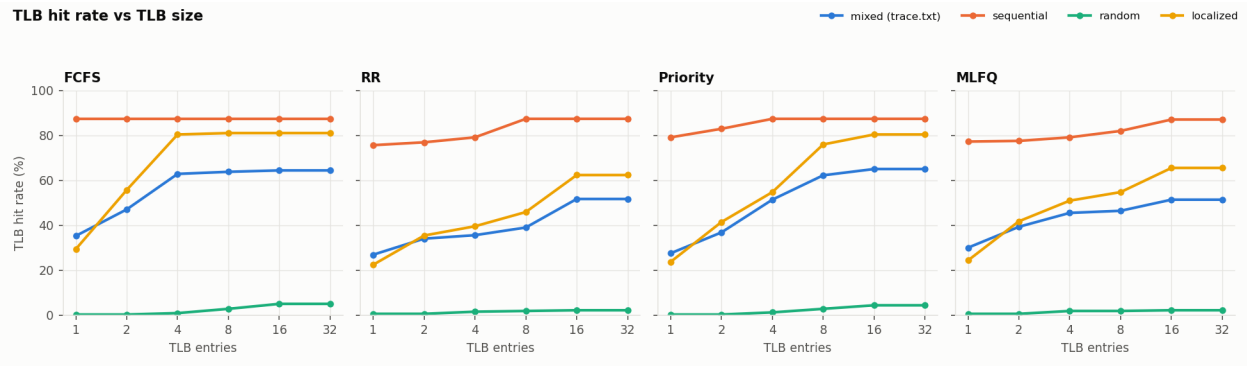


Figure 17: TLB hit rate as the TLB grows from 1 to 32 entries (16 frames).

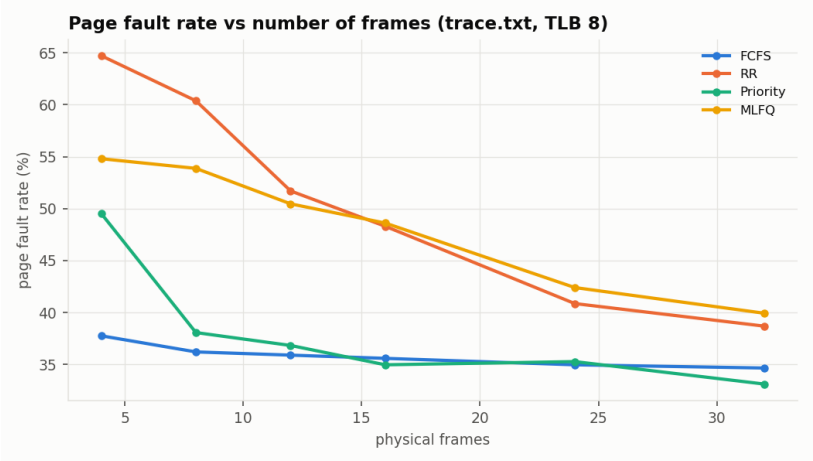


Figure 18: Page-fault rate as the number of physical frames grows (`trace.txt`).

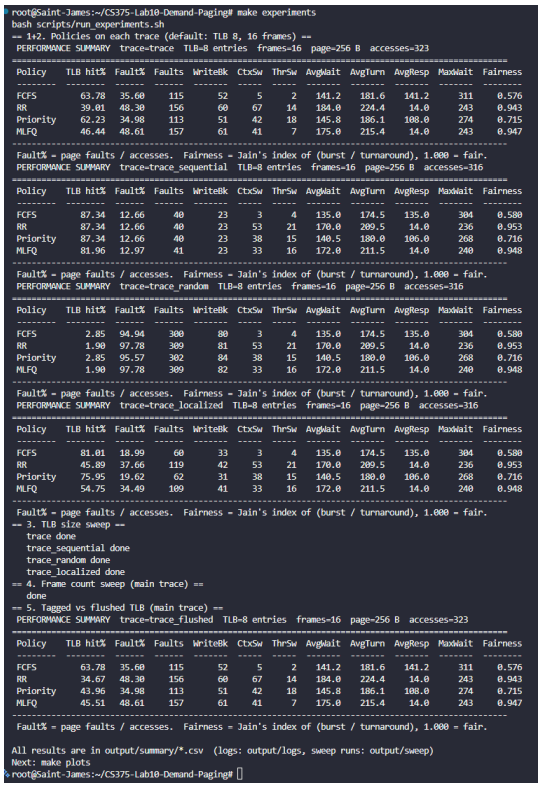


Figure 19: `make experiments`: summary tables for the sequential, random and localized traces.

5 Analysis

5.1 Scheduling policy vs. memory performance

TLB hit rate. FCFS has the best TLB hit rate on the mixed trace (63.8%), and RR has the worst (39.0%). Priority (62.2%) is close to FCFS, and MLFQ falls in between (46.4%). The explanation is the *reuse distance* between two accesses to the same page. Under FCFS a thread uses its pages in one uninterrupted burst, so its working set stays in the TLB. Under RR, all eight threads take turns every 4 ticks. By the time a thread runs again, seven other threads have each inserted one to three translations, and an 8-entry LRU TLB has evicted most of its entries. The RR timeline shows a miss or a fault at the start of almost every quantum. Priority behaves like FCFS because, within each priority level, only two or three threads compete. MLFQ starts as RR (quantum 4), but demoted threads get quanta of 8 and 16 and reuse their pages for longer.

Page-fault rate. The same effect appears one level lower. With 16 frames, FCFS and Priority fault on about 35% of accesses, while RR and MLFQ fault on about 48%. Interleaving all eight threads means their combined working sets (about 24 hot pages for the localized threads alone) compete for 16 frames at the same time, so Clock evicts pages that will be needed again a few quanta later. Faults also cause disk writes: RR needed 60 dirty write-backs and FCFS needed 52. The frame sweep shows that the RR/MLFQ disadvantage shrinks as memory grows, from 64.7% vs. 37.8% faults with 4 frames to 38.7% vs. 34.7% with 32. With enough memory, ordering matters much less, because what remains are compulsory faults.

Context-switch overhead. FCFS performs only 5 context switches and 2 thread switches. RR performs 67 context switches, more than 13 times as many, with one at almost every quantum boundary. Priority (42) and MLFQ (41) are in between. MLFQ saves switches through its longer lower-level quanta, and Priority saves them because only a few threads share a level. The difference between a *thread* switch and a *context* switch matters here. A thread switch keeps the address space, so a tagged TLB and resident pages stay useful. A context switch changes it. The tagged-versus-flushed experiment measures the cost directly: flushing the TLB on each context switch leaves FCFS unchanged (63.8%, only 5 switches), but lowers RR from 39.0% to 34.7% and Priority from 62.2% to 44.0%. PID-tagged TLBs exist precisely to avoid this cost.

Thread scheduling fairness. The fairness ranking is the reverse of the memory ranking. Jain's index is 0.576 for FCFS, 0.715 for Priority, and 0.943 and 0.947 for RR and MLFQ. Under FCFS, P4.T2 has a burst of only 12 ticks but finishes at tick 323 after waiting 311 ticks, because it arrived last. RR and MLFQ give every thread a first response within 28 ticks (average response time 14.0 versus 141.2 for FCFS). Priority starves the priority-3 threads: P3.T1 and P3.T2 wait 273 and 274 ticks and do not get the CPU until ticks 220 and 224. MLFQ combines a fast response for everyone with fewer switches than RR. Its 39 demotions and 29 aging promotions keep long CPU-bound threads such as P4.T1 (burst 80) from monopolising the CPU without starving anyone. The aging threshold is a real trade-off. At 15 ticks, MLFQ makes 57 context switches and behaves almost like RR. At 60 ticks or more it makes only 26 switches and its TLB hit rate rises to 51.7–53.6%, at a small cost in fairness (0.932–0.938).

Overall. No single policy wins on every metric. FCFS and Priority give the best TLB and page-fault rates, at a severe cost in response time and fairness. RR is the fairest but is the worst for the memory system. MLFQ is the best compromise: its fairness is as high as RR's, it makes 39% fewer context switches, and it has a better TLB hit rate. These measurements reproduce the classic tension between locality and interactivity.

5.2 Impact of memory access patterns

The access pattern matters more than the scheduler. With identical threads and the default configuration:

- **Sequential** access gives a TLB hit rate of 87.3% and 12.7% faults under FCFS, RR and Priority. Each 256-byte page is touched 8 times (stride 32), so 7 of every 8 accesses hit and only the first access to each new page faults. These are compulsory faults that no replacement policy can avoid. Because each thread uses only one page at a time, eight threads need only eight translations, which fit exactly in the 8-entry TLB, so even RR loses nothing. MLFQ is slightly lower (82.0%) because its 16-tick quanta cross two pages per turn and push out other threads' entries.

- **Random** access defeats both caches: 95–98% of accesses fault and the TLB hit rate is below 3%. With 256 possible pages per process and 16 frames, almost no page is resident when it is touched again. No policy and no reasonable TLB size helps; only a much larger memory would.

- **Localized** access is the most sensitive to scheduling. Under FCFS it is almost as good as sequential (81.0% hits, 19.0% faults), because each thread's 3-page hot set fits easily. Under RR the hit rate falls to 45.9% and faults double to 37.7%, because eight hot sets of 3 pages

(24 pages) do not fit in 16 frames or 8 TLB entries at the same time. Locality only pays off if the scheduler lets a thread run long enough to use it.

5.3 Impact of TLB size

The TLB-sweep graph shows three regimes:

1. **TLB smaller than the working set**: each doubling helps a lot. For localized/FCFS the hit rate goes 29.4% → 55.7% → 80.4% at 1, 2 and 4 entries, because a 3-page hot set needs about 4 entries.

2. **TLB covers the working set**: the curve flattens. For FCFS, going from 4 to 32 entries gains less than 2 percentage points.

3. **TLB larger than physical memory**: no gain at all. Every curve is identical at 16 and 32 entries, because a TLB can only map pages that are resident, and there are only 16 frames. Every miss beyond that point is a page fault. TLB hit rate + page-fault rate $\leq 100\%$, so RR on localized access can never exceed $100 - 37.7 = 62.3\%$, and it reaches exactly 62.3%.

Interleaving policies need a larger TLB to reach the same hit rate. RR and MLFQ on the mixed trace keep improving up to 16 entries (39.0% → 51.7%), while FCFS saturates at 4 entries. Random access never exceeds 5.1% at any size.

5.4 Conclusion

The simulator demonstrates demand paging with a two-level table, an LRU TLB and an enhanced Clock that avoids disk writes by preferring clean victims. Memory performance is a property of the whole system, not only of the MMU. The scheduling policy determines how much of each thread's locality survives. Long, uninterrupted bursts (FCFS, Priority, or MLFQ's lower queues) preserve TLB and frame contents. Frequent switching among many threads (RR) increases misses, faults and write-backs. A well-tuned MLFQ with aging gets most of RR's fairness at a much lower memory cost.

A Generated Files

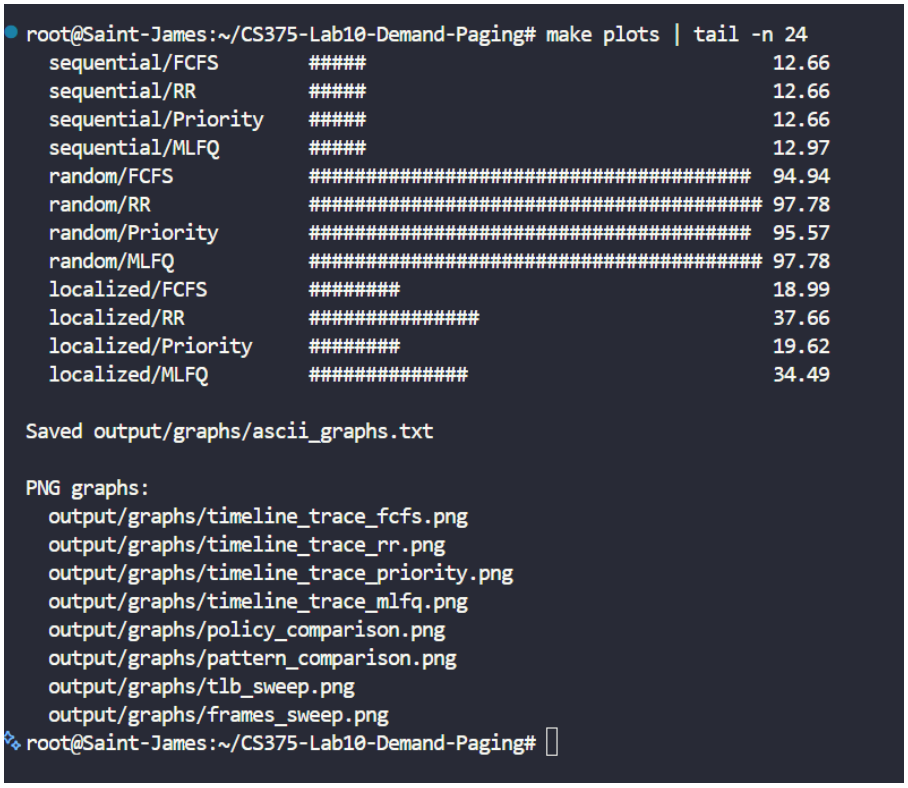


Figure 20: `python3 scripts/plotter.py`: the ASCII graphs and the list of generated PNG files.

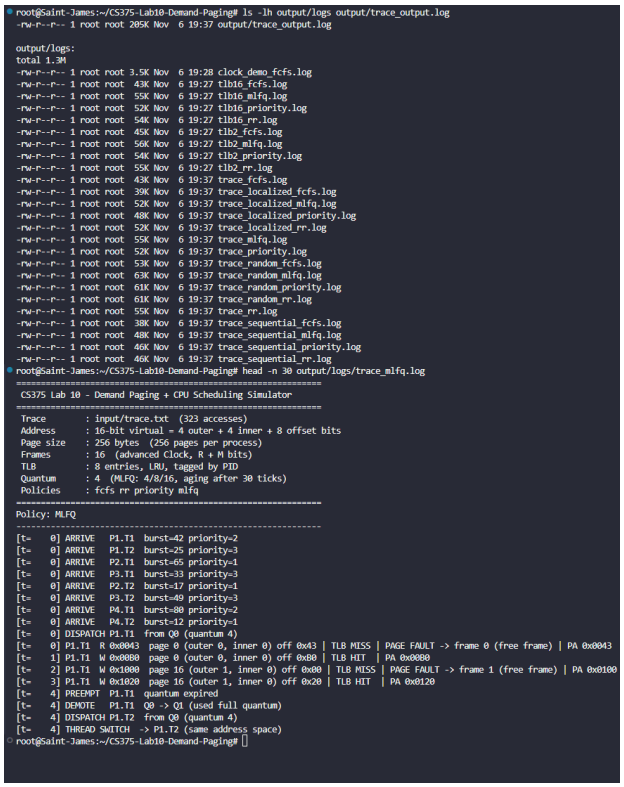


Figure 21: Auto-generated log files and the start of a log.

Path	Contents
`output/logs/<trace>_<policy>.log`	full per-tick log, statistics, frame table, ASCII timeline
`output/events/<trace>_<policy>.csv`	one row per tick (input to `plotter.py`)
`output/summary/<trace>.csv`	summary metrics, one row per policy
`output/summary/*_sweep.csv`	TLB-size and frame-count sweeps, tagged vs. flushed TLB
`output/graphs/*.png`	timeline and comparison graphs (plus `ascii_graphs.txt`)